

Alzikrayat Architecture Report

1. Overview

Alzikrayat is a photo sharing web application built to implement MVC architecture

and 3-tier separation without relying on a full stack web framework

The backend is written in Python using

Flask, business logic and data access are all implemented

2. 3-Tier Architecture & MVC Mapping

The system maintains a physical 3-tier separation, with each tier mapped to a logical MVC role:

Tier	MVC Role	Implementation
Presentation	Views	Jinja2 templates (app/views/templates/), Bootstrap 5 CSS
Application	Controllers	app/controllers/ — AuthController, PhotoController, CommentController
Data	Models	app/models/ — raw parameterized SQL via PyMySQL, no ORM

Requests enter through a single Flask catch-all view (app/routes.py:register_routes). Flask's own responsibility ends at listening for the HTTP request and handing off the path and method; from there, application-level dispatch is handled entirely by a hand-written component described in Section 3.

3-

Project structure

```
flask_project/
├── app/
│   ├── config.py
│   ├── routes.py
│   ├── core/
│   │   └── router.py
│   ├── controllers/
│   ├── models/
│   ├── views/templates/
│   └── static/
├── schema.sql
├── run.py
└── requirements.txt
```

4. Database Design

Three normalized MySQL tables, connected by foreign keys with cascading deletes so that removing a user or photo cleanly removes its dependent rows:

- users id, first_name, last_name, email (unique), password (bcrypt hash), description,
- photos id, user_id (FK → users, cascade), file_name, title, description, date_time
- comments id, photo_id (FK → photos, cascade), user_id (FK → users, cascade), comment, date_time

All queries are parameterized (PyMySQL %s placeholders) no string interpolation is used to build SQL which is the primary defense against SQL injection throughout the data layer.

5. Main features(Security and Validation)

- Passwords hashed with bcrypt before storage raw passwords are never stored.
- SQL injection every query uses parameterized placeholders no raw string formatting into SQL statements.
- XSS Jinja2 autoescapes all template output by default The custom mention highlight filter explicitly escapes comment text before injecting its own trusted markup so user submitted HTML can never execute.
- Three-layer validation HTML5 attributes (required, minlength, pattern), JavaScript (Bootstrap needs validation), and server side checks in each controller (length, format, uniqueness) before any database write.
- Ownership checks photo and comment deletion compare the requesting session's user id against the resource's stored user_id before any mutation is allowed.

6. Novelty Feature (Mention Autocomplete)

The comment box implements a live @ mention system as the user types @ followed by characters, client side JavaScript filters the full registered user list (passed to the template as JSON) and renders a drop down list of matching names selecting an entry inserts @FirstLast into the comment text on display the mention highlight Jinja2 filter uses a regular expression to find @word patterns in already escaped

comment text and wraps them in a styled span rendering mentions visually distinct without ever trusting raw user input as HTML.

In conclusion

Alzikrayat is a nearly completed web app all tested out end to end